

Relatório Técnico

Controlador de Vending Machine em SystemVerilog

Autores: João Pedro de Lima e Silva e Mateus Penha Albuquerque Pierre

Data: Julho de 2026

Repositório: <https://github.com/joaopedro-1/vending-machine-controller>

1. Diagrama de Blocos

O sistema é composto por duas unidades funcionais separadas: a unidade de controle (FSM) e o caminho de dados (datapath). O diagrama abaixo apresenta a arquitetura completa do controlador.

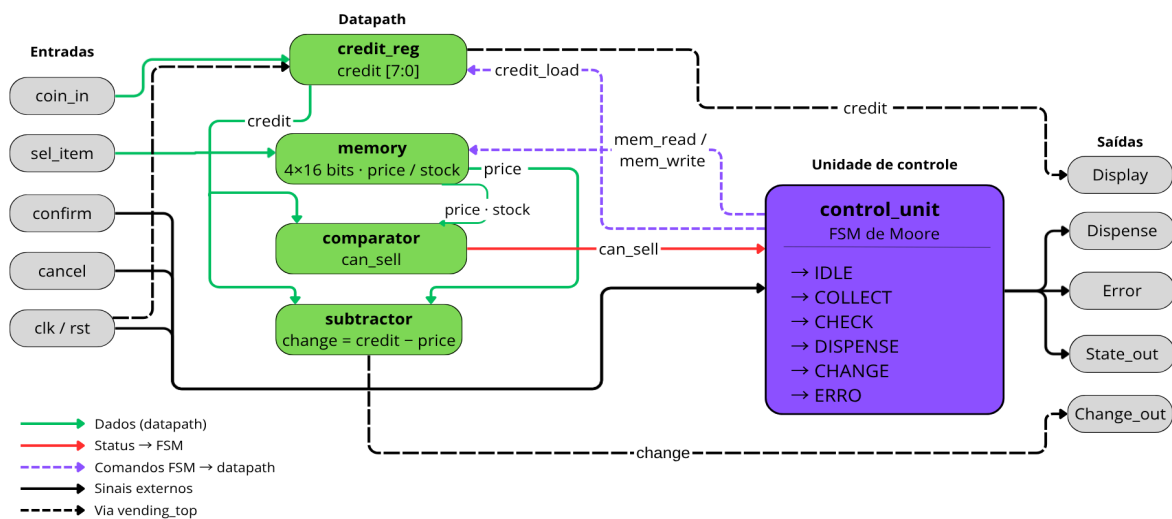


Figura 1 - Diagrama de Blocos

Entradas do sistema:

- **coin_in [1:0]** - Código da moeda inserida (00=nenhuma, 01=R\$0,25, 10=R\$0,50, 11=R\$1,00)
- **sel_item [1:0]** - Item selecionado (0=café, 1=água, 2=suco, 3=snack)
- **confirm** - Confirmação de compra pelo usuário
- **cancel** - Cancelamento e devolução do crédito
- **clk** - Clock global síncrono
- **rst** - Reset síncrono ativo em nível alto

Saídas do sistema:

- **display [7:0]** - Crédito acumulado atual
- **dispense** - Pulso de 1 ciclo para liberar o item
- **error** - Crédito insuficiente ou estoque zerado
- **state_out [2:0]** - Estado atual da FSM (debug)
- **change_out [7:0]** - Valor do troco em centavos

Sinais internos de interconexão:

- **credit** - Crédito acumulado (credit_reg → comparator, subtractor, display)
- **price / stock** - Preço e estoque lidos da memória (memory → comparator, subtractor)
- **can_sell** - Resultado do comparador (comparator → FSM)
- **change** - Troco calculado (subtractor → vending_top)
- **credit_load** - Autoriza acumulação de crédito (FSM → credit_reg)
- **mem_read / mem_write** - Leitura e escrita na memória (FSM → memory)
- **change_ena** - Autoriza captura do troco (FSM → vending_top)

2. Diagrama de Estados

A unidade de controle é uma FSM de Moore com 6 estados. As saídas dependem exclusivamente do estado atual, nunca das entradas.

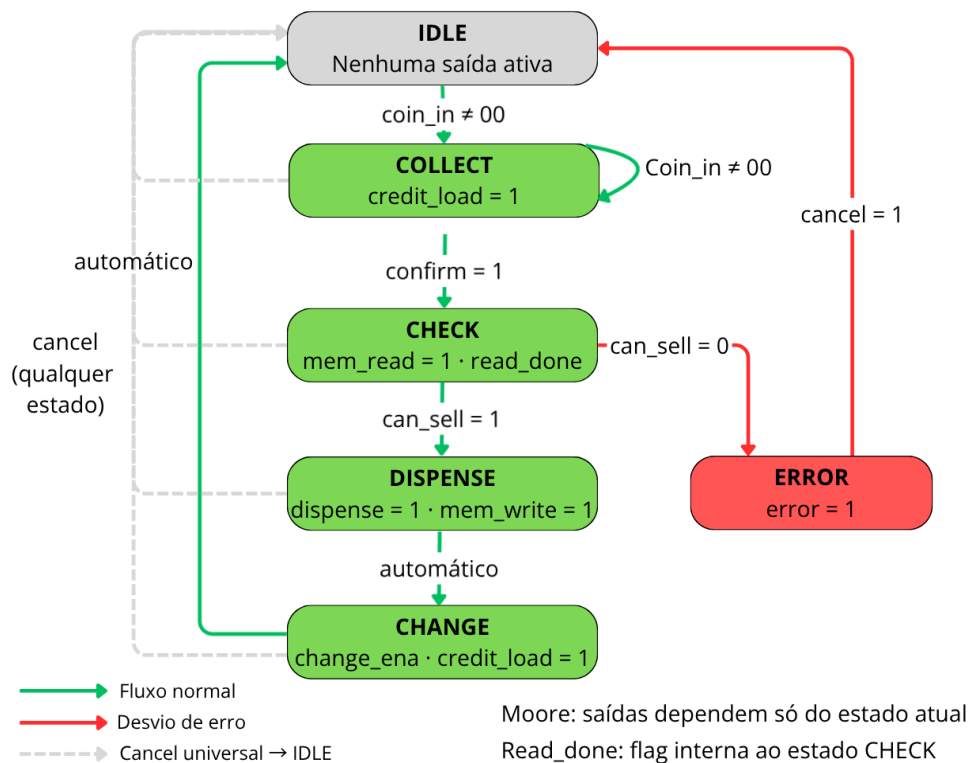


Figura 2 - Diagrama de Estados

Fluxo normal:

IDLE → COLLECT → CHECK → DISPENSE → CHANGE → IDLE

Desvio de erro:

CHECK → ERROR → IDLE (Via cancel)

Descrição dos estados:

Estado	Encoding	Saídas ativas	Condição de saída
IDLE	3'b000	nenhuma	coin_in ≠ 00
COLLECT	3'b001	credit_load = 1	confirm = 1
CHECK	3'b010	mem_read = 1	read_done = 1
DISPENSE	3'b011	dispense = 1, mem_write = 1	automático
CHANGE	3'b100	change_ena, credit_load = 1	automático
ERROR	3'b101	error = 1	cancel = 1

Observações:

- O sinal **cancel** retorna ao IDLE a partir de qualquer estado
- O estado CHECK utiliza a flag interna **read_done** para aguardar 1 ciclo após ativar **mem_read**, garantindo que os dados da memória síncrona estejam disponíveis antes de avaliar **can_sell**
- O estado DISPENSE dura exatamente 1 ciclo de clock, garantindo que **dispense** seja um pulso de 1 ciclo

3. Descrição dos Módulos RTL

O projeto é composto por 7 arquivos SystemVerilog organizados em hierarquia.

3.1 vending_pkg.sv

Package compartilhado por todos os módulos. Define o tipo **state_t** com o encoding dos 6 estados via **typedef enum logic [2:0]**, e os parâmetros de valor das moedas (**COIN_25**, **COIN_50**, **COIN_100**) e preços dos itens (**PRICE_CAFE**, **PRICE_AGUA**, **PRICE_SUCO**, **PRICE_SNACK**) em centavos.

3.2 credit_reg.sv

Registrador síncrono de 8 bits responsável por armazenar o crédito do usuário. No estado COLLECT, acumula o valor das moedas inseridas e, no estado CHANGE, zera o crédito após a devolução do troco. O registrador também é zerado por **reset** ou **cancel**. A utilização de **current_state** eliminou a necessidade de um sinal adicional para limpeza do registrador.

3.3 memory.sv

Memória síncrona de 4×16 bits que armazena preço e estoque dos quatro produtos. Os 8 bits mais significativos representam o preço e os 8 bits menos significativos armazenam o estoque. A leitura e a escrita são síncronas, sendo a escrita utilizada apenas para decrementar o estoque após uma venda.

Endereço	Item	Preço	Estoque inicial
0x0	Café	R\$0,25	5
0x1	Água	R\$0,50	5
0x2	Suco	R\$0,75	3
0x3	Snack	R\$1,00	2

3.4 comparator.sv

Módulo combinacional responsável por verificar se a venda pode ser realizada, comparando o crédito acumulado com o preço do produto e verificando a disponibilidade em estoque.

3.5 subtractor.sv

Módulo combinacional que calcula continuamente o troco (**credit - price**). O resultado é registrado apenas quando a FSM entra no estado CHANGE.

3.6 control_unit.sv

Unidade de controle implementada como uma FSM de Moore composta por dois blocos: **always_ff**, responsável pelo registro do estado atual, e **always_comb**, responsável pela lógica de transição de estados e geração das saídas. O tratamento do sinal cancel garante retorno imediato ao estado IDLE.

3.7 vending_top.sv

Módulo de topo responsável por interligar a FSM, memória e datapath. Também registra o valor do troco em **change_out** e disponibiliza o crédito acumulado na saída **display**.

4. Ambiente de Verificação

4.1 Estrutura do Testbench

O testbench foi desenvolvido em SystemVerilog sem utilização de UVM. A estrutura inclui geração de clock e reset, tarefas para inserção de moedas e realização de compras, verificação automática por meio de comparações entre valores esperados e obtidos e geração de waveforms em formato VCD para depuração.

4.2 Cenários de Teste

Cenário 1: Compra bem-sucedida com troco

Foi realizada a compra de um café (R\$ 0,25) utilizando uma moeda de R\$ 1,00. O teste verificou o acionamento do sinal **dispense**, a geração correta de 75 centavos de troco e a limpeza do registrador de crédito após a conclusão da venda.

Cenário 2: Crédito insuficiente

Foi inserido um crédito de R\$ 0,25 para a tentativa de compra de um snack (R\$ 1,00). O objetivo foi verificar a ativação do sinal **error**, a transição da FSM para o estado ERROR e o retorno ao estado IDLE após o cancelamento da operação.

Cenário 3: Cancelamento em COLLECT

Após a inserção de duas moedas de R\$ 1,00, a operação foi cancelada antes da confirmação da compra. O teste validou a devolução integral do crédito (R\$ 2,00), o zeramento do registrador de crédito e o retorno imediato da FSM ao estado IDLE.

Cenário 4: Estoque zerado

Foram realizadas cinco compras consecutivas de café, consumindo todo o estoque disponível. Na sexta tentativa de compra, foi verificada a ativação do sinal **error** e a transição da FSM para o estado ERROR, indicando indisponibilidade do produto.

4.3 Waveforms

As figuras a seguir apresentam as formas de onda capturadas no Synopsys WaveView para cada cenário de teste, mostrando os sinais **clk**, **state_out**, **display**, **coin_in**, **confirm**, **cancel**, **dispense**, **change_out** e **error**.

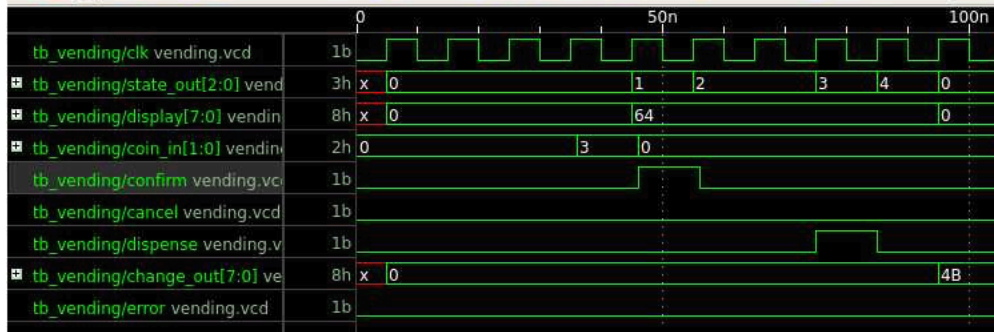


Figura 3 - Cenário 1: Compra bem-sucedida com troco

Sequência de estados: IDLE (0) → COLLECT (1) → CHECK (2) → DISPENSE (3) → CHANGE (4) → IDLE (0). Sinal dispense ativo por 1 ciclo durante DISPENSE. change_out = 0x4B (75 centavos) registrado em CHANGE. display mostra o crédito acumulado 0x64 (100 centavos) durante COLLECT.

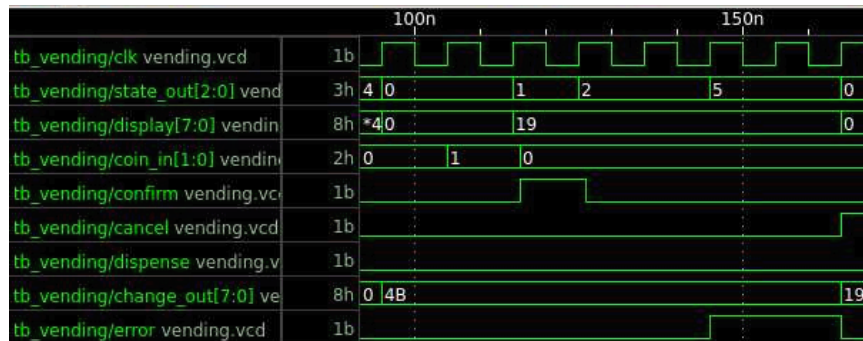


Figura 4 - Cenário 2: Crédito insuficiente

Sequência de estados: IDLE (0) → COLLECT (1) → CHECK (2) → ERROR (5). error ativo em nível alto durante o estado ERROR. FSM permanece em ERROR até cancel = 1, retornando ao IDLE (0).

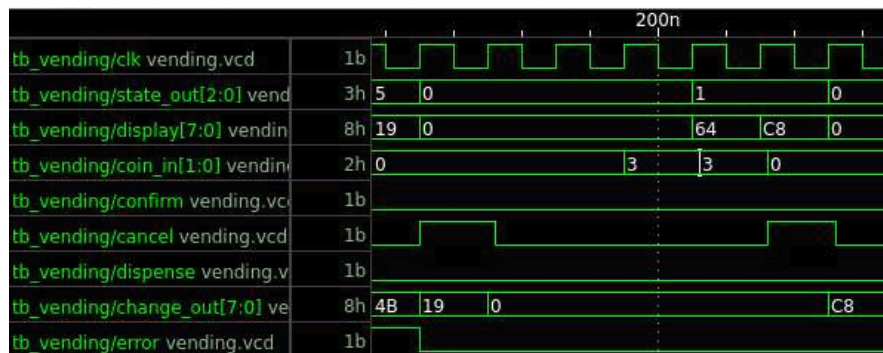


Figura 5 - Cenário 3: Cancelamento em COLLECT

Sequência de estados: IDLE (0) → COLLECT (1) → IDLE (0). display sobe até 200 centavos (0xC8) durante COLLECT (2×R\$1,00 inseridos). Após cancel = 1, FSM retorna imediatamente ao IDLE, crédito zerado e change_out = 200 centavos devolvidos.

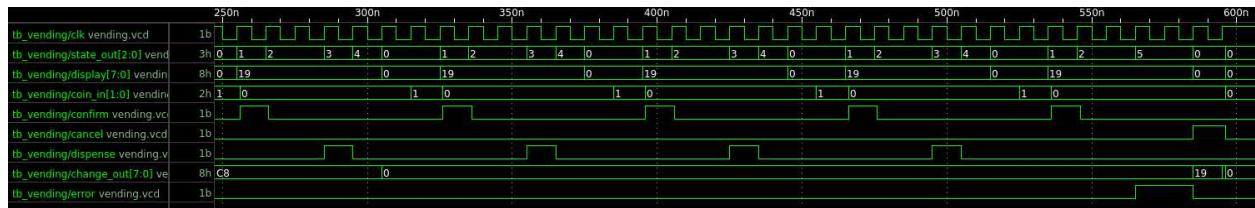


Figura 6 - Cenário 4: Estoque zerado após múltiplas compras

5 compras bem-sucedidas de café (estoque inicial = 5), cada uma com sequência IDLE (0) → COLLECT (1) → CHECK (2) → DISPENSE (3) → CHANGE (4) → IDLE (0). Na 6ª tentativa, can_sell = 0 pois estoque = 0, FSM vai para ERROR (5) e error é ativado.

4.4 Saída do Terminal

A seguir é apresentada a saída completa do testbench obtida no Synopsys VCS:

```
=== CENARIO 1: Compra bem-sucedida com troco (Cafe, R$0.25) ===
[PASS] Dispense deve disparar | Esperado: 1, Obtido: 1
[PASS] Troco deve ser 75 centavos | Esperado: 75, Obtido: 75
[PASS] Credito final deve ser 0 | Esperado: 0, Obtido: 0

=== CENARIO 2: Credito insuficiente (Snack, R$1.00) ===
[PASS] Sinal de erro deve ser ativado | Esperado: 1, Obtido: 1
[PASS] FSM deve travar no estado ERROR | Esperado: 5, Obtido: 5
[PASS] FSM deve voltar para IDLE apos cancel | Esperado: 0, Obtido: 0

=== CENARIO 3: Cancelamento em COLLECT ===
[PASS] FSM deve retornar a IDLE imediatamente | Esperado: 0, Obtido: 0
[PASS] Credito deve ser zerado | Esperado: 0, Obtido: 0
[PASS] Troco devolvido deve ser R$2.00 (200 centavos) | Esperado: 200, Obtido: 200

=== CENARIO 4: Estoque zerado apos multiplas compras (Cafe) ===
Tentando a 6a compra de Cafe (Estoque deve estar em 0)...
[PASS] Erro deve disparar por falta de estoque | Esperado: 1, Obtido: 1
[PASS] FSM deve ir para o estado ERROR | Esperado: 5, Obtido: 5

===== FIM DOS TESTES SIMULADOS =====
$finish called from file "sim/tb_vending.sv", line 140.
$finish at simulation time          596000
      V C S   S i m u l a t i o n   R e p o r t
Time: 596000 ps
CPU Time:      0.400 seconds;      Data structure size:  0.0Mb
Fri Jul 10 00:13:13 2026
[mateus.pierre@srv-microeletronica3 grupo_NN_vending]$
```

5. Síntese Lógica

A síntese lógica foi realizada com o Synopsys Design Compiler utilizando a biblioteca de células padrão SAED32 RVT (tt1p05v25c) com clock de 50 MHz (período de 20 ns).

5.1 Resultados de Área

Métrica	Valor
Células combinacionais	150
Células sequenciais	60
Área combinacional	372,58 μm^2
Área não-combinacional	396,46 μm^2
Área total de células	769,04 μm^2
Área total (com interconexão)	908,76 μm^2

A soma das áreas dos módulos corresponde à área total de células (769,04 μm^2). A área total do circuito (908,76 μm^2) é superior por incluir a estimativa de área de interconexão realizada pelo Design Compiler.

Distribuição hierárquica:

Módulo	Área (μm^2)	% do total
memory	471,18	61,3%
credit_reg	141,56	18,4%
control_unit	68,87	9,0%
comparator	7,62	1,0%
subtractor	0,00	0,0%
vending_top (lógica local)	79,80	10,4%

A memória representa o maior bloco do design, pois armazena registradores para os 4 itens com preço e estoque. O subtrator foi completamente otimizado pelo Design Compiler por ser lógica combinacional pura sem restrição de timing crítico.

5.2 Resultados de Timing

Métrica	Valor
Período do clock	20 ns (50 MHz)
Clock uncertainty	0,5 ns
Atraso do caminho crítico	0,16 ns
Slack	15,60 ns
Violações de setup	nenhuma
Violações de hold	nenhuma

5.3 Resultados de Potência

Métrica	Valor
Potência interna	20,37 μ W
Potência de switching	0,39 μ W
Potência dinâmica total	20,76 μW
Potência de leakage	87,62 μW
Potência total	108,39 μW

6. Análise

6.1 Caminho Crítico

O caminho crítico identificado pelo Design Compiler percorre:

coin_in[0] (entrada) → somador de 8 bits → **credit_reg[7]** (registrador)

O caminho está associado ao estado COLLECT, onde o crédito é atualizado (**credit** ← **credit** + **coin_value**). Um somador de 8 bits com propagação de carry (ripple-carry) é a operação com maior número de portas em série de todo o projeto — mais profunda até do que a lógica de decisão da FSM, que é proporcionalmente pequena e rápida.

6.2 Frequência Máxima de Operação

Com base na exploração de timing realizada, a frequência máxima de operação do design é estimada entre 200 MHz e 250 MHz, o último período com slack não-negativo foi de 5 ns (200 MHz, slack = +0,60 ns) e o período de 4 ns (250 MHz) violou o timing com slack de -0,01 ns.

Fmax estimado ≈ 200 MHz

6.3 Exploração dos Limites de Timing

A tabela abaixo apresenta os resultados da síntese para diferentes períodos de clock:

Período (ns)	Frequência (MHz)	Slack (ns)	Área (µm²)	Timing
20	50,00	+15,60	908,76	MET
16	62,50	+11,60	908,76	MET
12	83,33	+7,60	908,76	MET
8	125,00	+3,60	908,76	MET
6	166,67	+1,60	908,76	MET
5	200,00	+0,60	908,76	MET
4	250,00	-0,01	1076,26	VIOLATED

Observações:

- O slack diminui linearmente à medida que o período é reduzido que é o padrão esperado para um design com caminho crítico curto e fixo
- A área permanece constante até 5 ns, o DC não precisa reestruturar o circuito para fechar o timing
- No período de 4 ns o DC violou o timing com slack de apenas -0,01 ns e aumentou a

- área para 1076,26 μm^2 tentando encontrar uma solução alternativa
- O período mínimo suportado é 5 ns (200 MHz)

6.4 Distribuição de Área

A memória representa 61,3% da área total que é o maior bloco do design por armazenar registradores síncronos para os 4 itens com preço e estoque. O subtrator apresenta área zero porque o DC otimizou completamente a lógica combinacional pura. A unidade de controle representa apenas 9% da área total, demonstrando que a FSM de Moore com 6 estados é eficiente em área.

6.5 Discussão

A separação entre unidade de controle e datapath contribuiu para um caminho crítico curto, resultando em operação sem violações até 200 MHz. O uso da flag **read_done** permitiu acomodar a latência da memória síncrona sem aumentar o número de estados da FSM. Quando o período foi reduzido para 4 ns, o Design Compiler aumentou a área do circuito tentando atender às restrições temporais, porém sem sucesso devido ao atraso do somador de 8 bits presente no registrador de crédito.

7. Conclusão

O projeto permitiu aplicar todas as etapas do fluxo de desenvolvimento RTL, incluindo especificação, implementação em SystemVerilog, verificação funcional com Synopsys VCS e síntese lógica utilizando o Design Compiler. Os resultados mostraram que o circuito atende aos requisitos funcionais e temporais, operando com folga à frequência de projeto de 50 MHz e alcançando frequência máxima próxima de 200 MHz. A análise dos relatórios evidenciou que o caminho crítico está associado ao somador do registrador de crédito, reforçando a importância da análise de timing e da otimização arquitetural em projetos digitais.

A maior dificuldade foi identificar o período mínimo em que o timing ainda fecha. Rodamos a síntese em diferentes períodos (20, 16, 12, 8, 6, 5 e 4 ns), registrando slack e área a cada execução, até identificar que o limite ficava entre 5 ns (ainda fechando) e 4 ns (já violando).

Percebemos que área e timing não variam de forma simples: entre 20 ns e 5 ns a área não mudou nada, mas em 4 ns o Design Compiler tentou compensar aumentando a área em quase 20%, e mesmo assim não fechou o timing mostrando que existe um limite físico da lógica que só reduzindo a profundidade do caminho crítico (por exemplo, um somador mais rápido) resolveria.

Tivemos que organizar bem os relatórios de cada rodada (área, timing, power, constraint) para não misturar os resultados de períodos diferentes na hora de montar a tabela.